

DAY 4 - BUILDING DYNAMIC FRONTEND

COMPONENTS

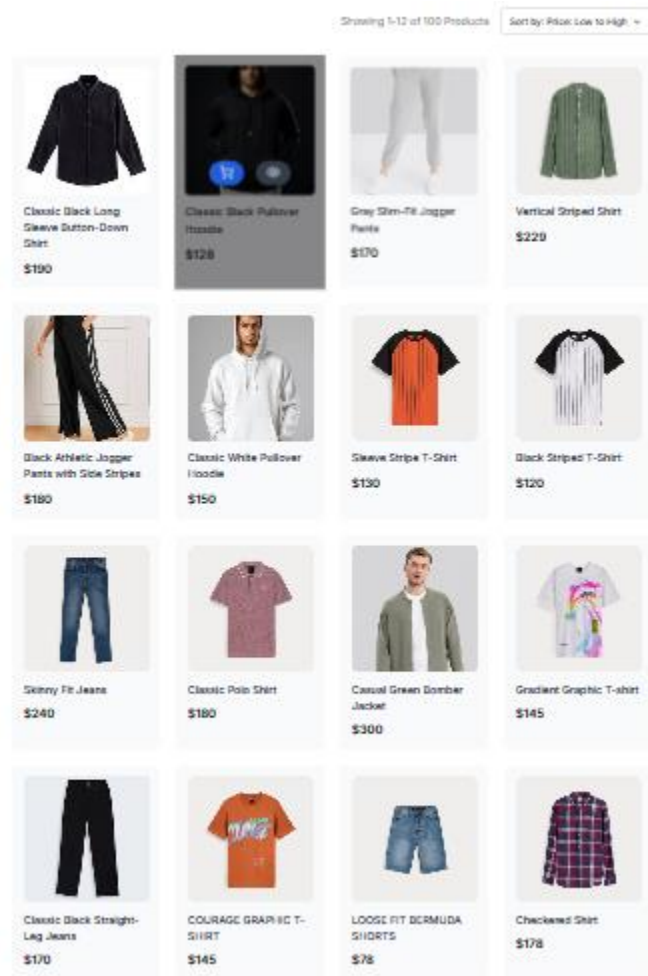
(SHOP.CO)

Day 4 Activity Summary:

On Day 4, we focused on building dynamic frontend components using Sanity CMS and APIs, emphasizing reusable and modular design. State management techniques were implemented for efficient data handling. We also practiced responsive design and UX/UI best practices, preparing for real-world client projects with professional workflows.

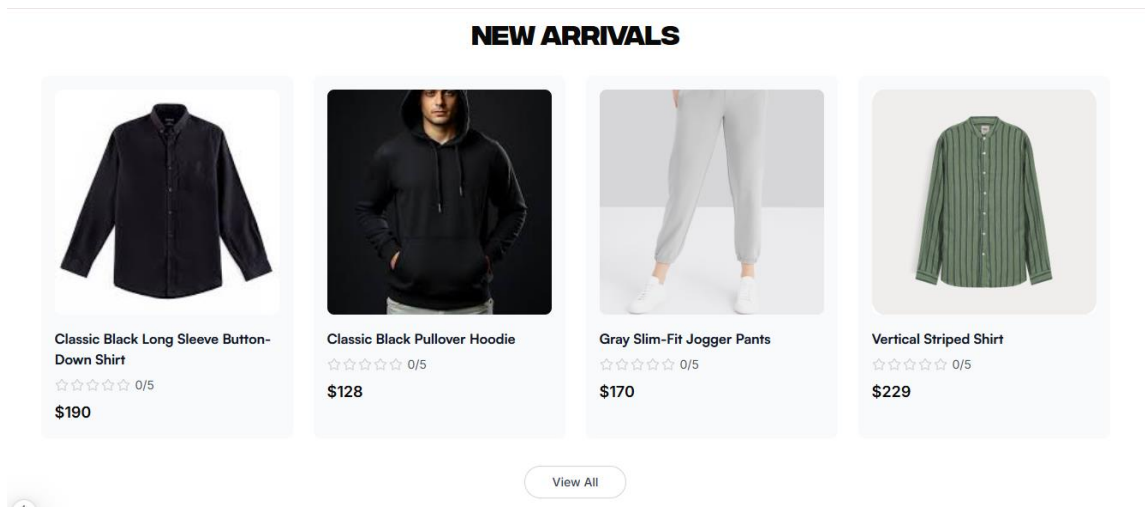
Product Listing:

I fetched product data for the listing using a sanity query and displayed it on the UI. By using Groq queries, I was able to pull accurate and relevant product information to show in a clear and organized way on the website.



New Arrivals:

For the "New Arrivals" section, I used a Groq query to fetch the latest product data from Sanity. This allowed me to display newly added products on the UI, showcasing fresh arrivals with their images, descriptions, and prices in a user-friendly format.



here's a code:

```

app > product > [id] > page.tsx > client
1  'use client';
2
3  import React, { useEffect, useState } from 'react';
4  import { useParams } from 'next/navigation';
5  import { useCart } from '@context/CartContext';
6  import { toast } from 'react-hot-toast';
7  import sanityClient from '@sanity/client';
8  import Testimonials from '@components/home/testimonials';
9  import NewArrivals from '@components/home/top-selling';
10
11
12  const client = sanityClient({
13    projectId: 'vc3rco8c',
14    dataset: 'production',
15    useCdn: true,
16  });
17
18  export default function ProductDetailPage() {
19    const { id } = useParams();
20    const { dispatch } = useCart();
21    const [product, setProduct] = useState<any>(null);
22    const [loading, setLoading] = useState(true);
23    const [size, setSize] = useState<string>(''); // Selected size
24    const [color, setColor] = useState<string>(''); // Selected color
25    const [quantity, setQuantity] = useState<number>(1); // Selected quantity
26    const [selectedImage, setSelectedImage] = useState<string>(''); // Selected image
27
28    // Fetch product details
29    useEffect(() => {
30      const fetchProduct = async () => {
31        try {
32          const fetchedProduct = await client.fetch(
33            `*[_id == $id][0]{

```

Product Details and Dynamic Routing:

I fetched product details for the listing using a sanity query with Groq. The data was then displayed on the UI, including product images, descriptions, prices, and key features, ensuring the information was accurate and well-organized for the users.

Product 1:

SHOP.CO

Shop

On Sale

New Arrivals

Brands

Search for products...



Vertical Striped Shirt

This product is a stylish vertical striped shirt, featuring alternating green and darker green stripes for a sophisticated, yet casual look. The shirt has a simple button-down design with a modern band collar, offering a clean and sharp appearance. Its long sleeves and breathable fabric make it an ideal choice for various occasions, from a laid-back day out to semi-casual gatherings. The subtle stripe pattern adds a unique touch while maintaining versatility, allowing it to pair well with both jeans and chinos.

☆☆☆

\$229

Size

Small

Color

Red

Quantity:

-

1

+

Add to Cart

Product 2:



Classic Black Pullover Hoodie

This versatile and stylish black hoodie is a must-have in any wardrobe. Featuring a comfortable, soft fabric and a relaxed fit, it is perfect for casual outings or layering for colder weather. The hoodie includes an adjustable drawstring and a spacious front pocket, making it both practical and fashionable. The black color adds a timeless touch, making it easy to pair with jeans, shorts, or joggers for an effortlessly cool look.

☆☆☆

\$128

Size

Small

Color

Red

Quantity:

-

1

+

Add to Cart

Product 3:



Casual Green Bomber Jacket

This stylish green bomber jacket offers a sleek and modern twist on a classic design. Made from soft and comfortable fabric, it features snap buttons and ribbed cuffs, giving it a sporty yet refined look. The minimalist style makes it perfect for layering over casual t-shirts or hoodies. Whether you're out with friends or just lounging, this jacket provides a laid-back yet fashionable vibe. Its muted green color adds a subtle, earthy tone that pairs well with a variety of outfits, making it a versatile addition to your casual wardrobe.

☆☆☆

\$300

Size

Small

Color

Red

Quantity:

-

1

+

Add to Cart

Product 4:



LOOSE FIT BERMUDA SHORTS

These Loose Fit Bermuda Shorts are designed for comfort and laid-back style. Made from lightweight, breathable fabric, these shorts offer a relaxed fit that hits just above the knee. The versatile design features a classic waistband with belt loops, side pockets, and a simple yet stylish look perfect for casual outings or beach days. Pair them with your favorite t-shirt or tank for a cool and comfortable summer outfit.

☆☆☆

\$78

Size

Small

Color

Red

Quantity:

-

1

+

Add to Cart

Categories:

For the category components, I fetched product data from Sanity using Groq queries to display items based on their categories. This allowed me to organize products into specific sections like clothing, shoes, and accessories on the UI, making it easier for users to browse and find what they're looking for.

Shop ^

On Sale

New Arrivals

Brands

Q Search for products

Men's clothes

In attractive and spectacular colors and designs

Formal Style

For all ages, with happy and beautiful colors

Women's clothes

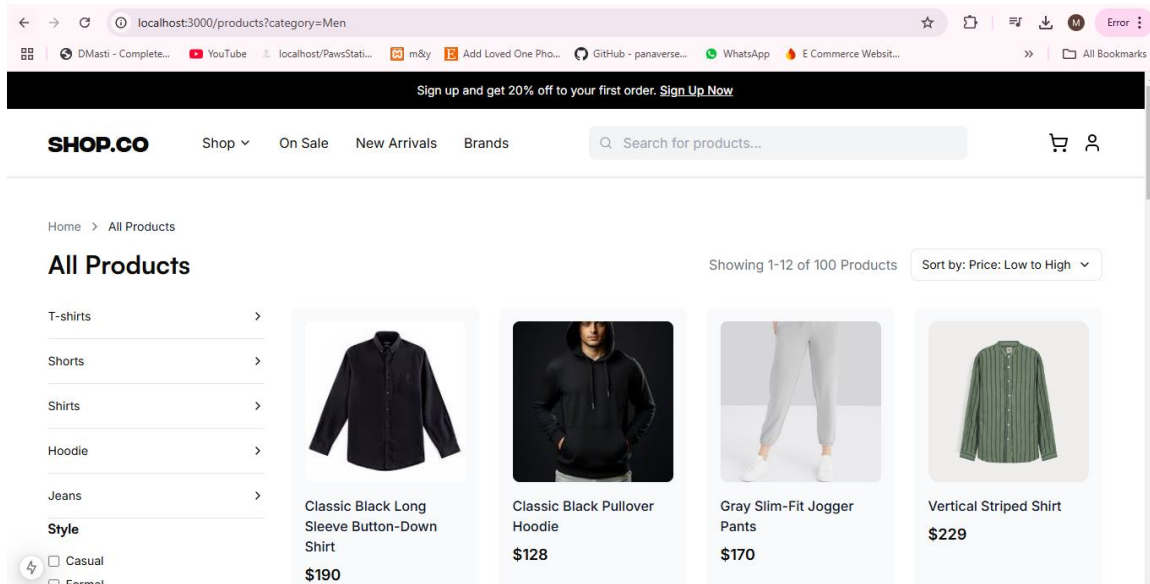
Ladies, your style and tastes are important to us

Casual

Suitable for men, women and all tastes and styles

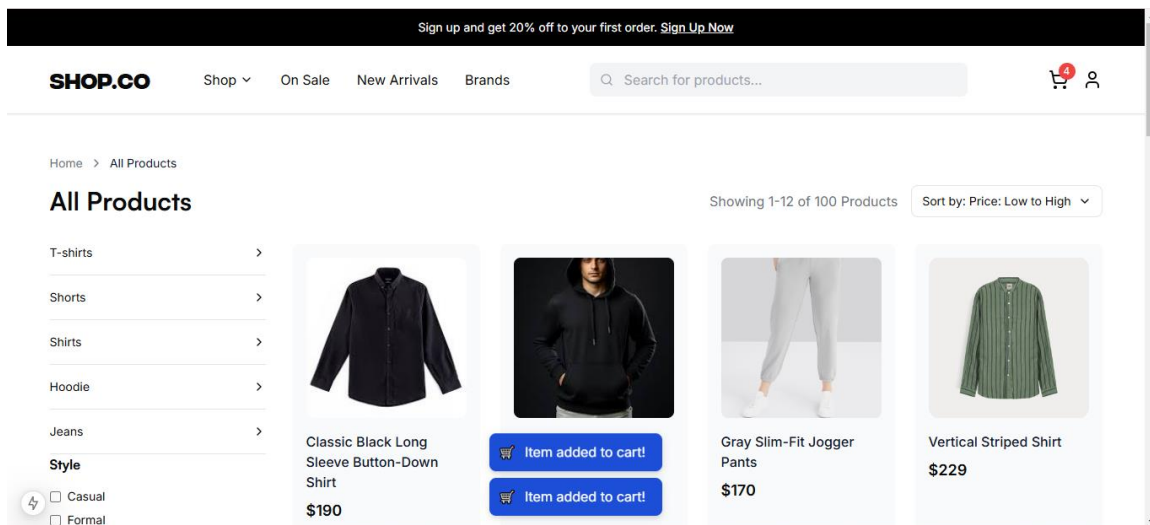
DISCOUNT
AT
OUR STYLE

```
export const menuItems = [
  {
    title: "Men's clothes",
    subtitle: "In attractive and spectacular colors and designs",
    href: "/products?category=Men"
  },
  {
    title: "Women's clothes",
    subtitle: "Ladies, your style and tastes are important to us",
    href: "/products?category=Women"
  },
  {
    title: "Formal Style",
    subtitle: "For all ages, with happy and beautiful colors",
    href: "/products?style=formal"
  },
  {
    title: "Casual",
    subtitle: "Suitable for men, women and all tastes and styles",
    href: "/products?style=casual"
  }
]
```



Cart Component:

For the cart components, I implemented a system that fetches and manages cart data, allowing users to add and remove products. Using Groq queries and the Sanity database, I ensured the cart content was accurately displayed on the UI, with product details like name, price, and quantity, and updated dynamically as users interacted with it.



CODE:

```

1  'use client';
2
3  import { ShoppingCart, Eye } from 'lucide-react';
4  import { useState, useEffect } from 'react';
5  import { useCart } from '@context/CartContext';
6  import { CartItem } from '@types/cart';
7  import { toast } from 'react-hot-toast';
8  import sanityClient from '@sanity/client';
9  import Link from 'next/link';
10
11  const client = sanityClient({
12    projectId: "vc3nco0c", // Replace with your project ID
13    dataset: 'production', // Replace with your dataset name
14    useCdn: true, // Use CDN for faster queries
15  });
16
17  interface Product {
18    _id: string;
19    name: string;
20    price: number;
21    originalPrice?: number;
22    rating: number;
23    image: {
24      asset: {
25        url: string;
26      };
27    };
28  };
29
30  export default function ProductCard({ product }: { product: Product }) {
31    const { dispatch, state } = useCart();
32    const [isClient, setIsClient] = useState(false);
33    const [loading, setLoading] = useState(false);

```

Order Summary:

For the order summary, I fetched data from the cart and order details using Groq queries from Sanity. The summary displayed key information such as the total number of items, individual product prices, taxes, shipping costs, and the final total, all updated in real-time as users proceed through the checkout process.

[Home](#) > [Cart](#)

YOUR CART



Classic Black Long Sleeve Button-Down Shirt

Size: M

Color: Red

- 28 +

\$190



Classic Black Pullover Hoodie

Size: M

Color: Red

- 7 +

\$128

Order Summary

Subtotal \$6216.00

Discount -\$87.02

Delivery Fee \$15.00

Total \$6143.98

Add promo code

Apply

Go to Checkout →

CODE:


```

interface OrderSummaryProps {
  subtotal: number
  discount: number
  deliveryFee: number
  total: number
}

export function OrderSummary({ subtotal, discount, deliveryFee, total }: OrderSummaryProps) {
  const formatPrice = (price: number) => price.toFixed(2)

  return (
    <div className="bg-gray-50 rounded-2xl p-6 lg:p-8">
      <h2 className={cn("text-xl font-medium mb-6", satoishi.className)}>Order Summary</h2>
      <div className="space-y-4">
        <div className="flex justify-between">
          <span className="text-gray-600">Subtotal</span>
          <span className="font-medium">${formatPrice(subtotal)}</span>
        </div>
        <div className="flex justify-between">
          <span className="text-gray-600">Discount</span>
          <span className="text-red-500">-${formatPrice(discount)}</span>
        </div>
        <div className="flex justify-between">
          <span className="text-gray-600">Delivery Fee</span>
          <span className="font-medium">${formatPrice(deliveryFee)}</span>
        </div>
        <div className="h-px bg-gray-200 my-4" />
        <div className="flex justify-between text-lg font-medium">
          <span>Total</span>
          <span>${formatPrice(total)}</span>
        </div>
      </div>
    </div>
  )
}

```

Checkout Component:

For the checkout component, I integrated the product data from the cart and user information using Groq queries from Sanity. This allowed users to review their order details, enter shipping and payment information, and finalize their purchase. The checkout process was designed to be seamless, with real-time updates on pricing, discounts, and shipping options.

Checkout

Shipping Information

Name

John Doe

Email

john@example.com

Phone

1234567890

Address

123 Main St

City

New York

State

NY

Zip Code

10001

Place Order

Order Summary

Classic Black Long Sleeve Button-Down Shirt (x28)	\$5320.00
Classic Black Pullover Hoodie (x7)	\$896.00
Total	\$6216.00

CUSTOMER FEEDBACK

OUR HAPPY CUSTOMERS



Sarah M.

"I'm blown away by the quality and style of the clothes I received from Shop.co. From casual wear to elegant dresses, every piece I've bought has exceeded my expectations."



Alex K.

"Finding clothes that align with my personal style used to be a challenge until I discovered Shop.co. The range of options they offer is truly remarkable, catering to a variety of tastes and occasions."



James L.

"As someone who's always on the lookout for unique fashion pieces, I'm thrilled to have stumbled upon Shop.co. The selection of clothes is not only diverse but also on-point with the latest trends."

FILTER:

All Products

T-shirts >

Shorts >

Shirts >

Hoodie >

Jeans >

Style

- ☐ Casual
- ☐ Formal
- ☐ Party
- ☐ Gym

Price



Apply Price Filter

RELATED PRODUCTS:

TOP SELLING



Classic White Pullover Hoodie

☆☆☆☆☆ 0/5

\$150



Sleeve Stripe T-Shirt

☆☆☆☆☆ 0/5

\$130



Black Striped T-Shirt

☆☆☆☆☆ 0/5

\$120



Skinny Fit Jeans

☆☆☆☆☆ 0/5

\$240

View All

BROWSE BY DRESS STYLE

Casual

Formal

Party

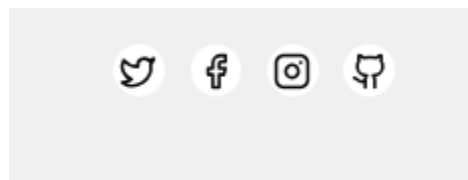
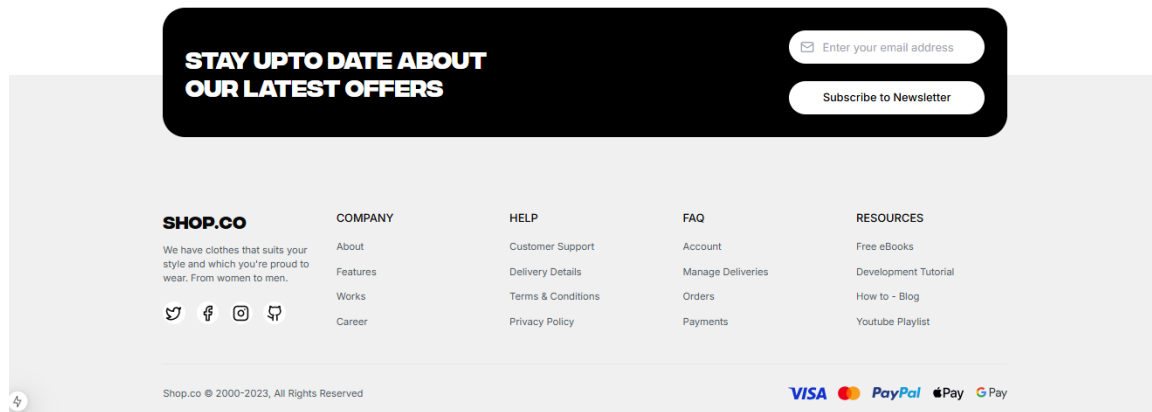
Gym

NAVBAR:

Sign up and get 20% off to your first order. [Sign Up Now](#)

SHOP.CO Shop ▾ On Sale New Arrivals Brands  

FOOTER:



SOCIAL MEDIA:

SCRIPTS CODE:

```
scripts > importData.mjs > ...
1  import { createClient } from '@sanity/client';
2
3  const client = createClient({
4    projectId: "vc3hrc0c",
5    dataset: "production",
6    useCdn: true,
7    apiVersion: '2025-01-13',
8    token: "sk6dstRbP4HjcUXlVvBrbteEKQTCTxgp7Nk87Tre5P8i8ghdivIDoVG3yXAfRtBxaIXT00cNkj6N2DZYe8VpUkw3K9C59MGi4FpfuvjZjnfpr2ftLnpodvVZw9MuHdGSMg159AZ"
9  });
10
11  async function uploadImageToSanity(imageUrl) {
12    try {
13      console.log(`Uploading image: ${imageUrl}`);
14
15      const response = await fetch(imageUrl);
16      if (!response.ok) {
17        throw new Error(`Failed to fetch image: ${imageUrl}`);
18      }
19
20      const buffer = await response.arrayBuffer();
21      const bufferImage = Buffer.from(buffer);
22
23      const asset = await client.assets.upload('image', bufferImage, {
24        filename: imageUrl.split('/').pop(),
25      });
26
27      console.log(`Image uploaded successfully: ${asset._id}`);
28      return asset._id;
29    } catch (error) {
30      console.error(`Failed to upload image:`, imageUrl, error);
31      return null;
32    }
33  }
```

